



Forwarding Básico y Fragmentación IP

Integrante: Igor Assis Passos de Souza, Nelson A. Navarro B.

Profesores: Ivana Bachmann

Fecha de realización: 21 de Junio de 2026

Fecha de entrega: 21 de Junio de 2026

Santiago, Chile

Índice

1. Introducción	1
2. Parte I: Forwarding Básico	1
2.1. Tabla de Rutas: Similitudes y Diferencias	1
2.1.1. Similitudes con una tabla de rutas real	1
2.1.2. Diferencias con una tabla de rutas real	1
2.2. Implementación de Round-Robin	1
2.2.1. Descripción de la implementación	1
2.2.2. Decisiones de diseño	2
2.3. Router Default	2
2.3.1. Tabla de rutas con router default	2
2.4. Pruebas: Mini-Internet sin TTL	2
2.4.1. Prueba 1: Tabla de rutas mal configurada	2
2.4.2. Prueba 2: Round-robin en red de 5 routers	3
2.4.3. Prueba 3: Round-robin en red extendida (R0 y R6)	3
2.5. Pruebas: Mini-Internet con TTL	3
2.5.1. Prueba 1: Tabla mal configurada con TTL=10	3
2.5.2. Prueba 2: Orden de llegada de paquetes	4
3. Parte II: Fragmentación IP	4
3.1. Prueba 1: Validación de TTL y Round-Robin	4
3.2. Prueba 2: Fragmentación y Re-ensamblaje	4
3.2.1. Comportamiento observado desde R1 a R5	5
3.2.2. Cambio de comportamiento según router origen	5
3.2.3. Capacidad de re-fragmentación	5
4. Conclusiones	6
4.1. Forwarding Básico	6
4.2. Fragmentación IP	6
4.3. Reflexión Final	6

1. Introducción

Este informe presenta el desarrollo e implementación de dos actividades fundamentales en el estudio de redes de computadores: Forwarding Básico y Fragmentación IP. Ambas actividades consisten en la simulación de un mini-Internet utilizando sockets UDP en Python, donde se implementan routers que pueden recibir, procesar y reenviar paquetes IP según tablas de rutas predefinidas.

La primera actividad, Forwarding Básico, se centra en el mecanismo fundamental de enrutamiento de paquetes en una red, implementando conceptos como Time-To-Live (TTL), round-robin para balanceo de carga y routers por defecto. La segunda actividad, Fragmentación IP, extiende la implementación anterior añadiendo el concepto de Maximum Transmission Unit (MTU) y la capacidad de fragmentar y reensamblar datagramas IP cuando estos exceden el tamaño máximo permitido por un enlace.

2. Parte I: Forwarding Básico

2.1. Tabla de Rutas: Similitudes y Diferencias

2.1.1. Similitudes con una tabla de rutas real

Las tablas de rutas utilizadas en esta actividad comparten varios conceptos fundamentales con las tablas de rutas reales:

- **Network Destination:** Ambas utilizan el concepto de red de destino para determinar hacia dónde dirigir un paquete.
- **Gateway/Next Hop:** Tanto en nuestra implementación como en tablas reales, se especifica la dirección del siguiente salto para llegar al destino.
- **Consulta secuencial:** Las rutas se revisan en orden hasta encontrar una que coincida con el destino del paquete.
- **Ruta por defecto:** Se implementa el concepto de gateway por defecto para paquetes dirigidos a destinos fuera de la red conocida.

2.1.2. Diferencias con una tabla de rutas real

Las principales diferencias se deben a las simplificaciones necesarias para fines pedagógicos:

- **Uso de puertos en vez de rangos IP:** En nuestra simulación, utilizamos rangos de puertos (ej: 8881-8883) para representar diferentes segmentos de red, mientras que una tabla real utiliza rangos de direcciones IP con notación CIDR.
- **Simulación local:** Toda nuestra red opera sobre la dirección IP 127.0.0.1 (localhost), utilizando diferentes puertos para simular distintos routers. En una red real, cada router tiene su propia dirección IP única.
- **Formato simplificado:** Nuestra tabla tiene un formato fijo de 5 columnas (Red, Puerto_Inicial, Puerto_Final, IP_Next_Hop, Puerto_Next_Hop), mientras que las tablas reales incluyen información adicional como métricas, interfaces de red y máscaras de subred.
- **Sin métricas de costo:** No implementamos conceptos como costo de ruta o preferencia entre rutas basada en métricas, características presentes en protocolos de enrutamiento reales como OSPF o RIP.

2.2. Implementación de Round-Robin

2.2.1. Descripción de la implementación

Para manejar múltiples rutas posibles hacia un mismo destino, se implementó un algoritmo de round-robin que alterna entre las rutas disponibles de forma cíclica. La implementación utiliza las siguientes estructuras:

Estructura de datos: Se utiliza un diccionario global `round_robin_state` que mantiene el estado de las rutas visitadas. La llave del diccionario es una tupla (`dest_ip`, `dest_port`) que identifica de forma única cada destino, y el valor es un contador que indica la próxima ruta a utilizar.

```
round_robin_state = {}  
route_key = (dest_ip, dest_port)  
idx = round_robin_state.get(route_key, 0) % len(matching_routes)  
round_robin_state[route_key] = idx + 1
```

Funcionamiento: Cuando se consultan las rutas para un destino específico:

1. Se buscan todas las rutas que coinciden con el puerto de destino en las tablas de rutas.
2. Se construye la llave única para el destino.
3. Se obtiene el índice actual del contador (o 0 si es la primera vez).
4. Se selecciona la ruta usando el módulo del contador con el número total de rutas.
5. Se incrementa el contador para la próxima consulta.

Manejo de múltiples áreas: El diseño basado en tuplas (`dest_ip`, `dest_port`) como llave permite que el router mantenga estados independientes de round-robin para diferentes áreas de la red. Por ejemplo, si el router R2 puede enviar paquetes tanto a la zona R1 como a la zona R5 por múltiples caminos, mantendrá contadores separados para cada destino.

2.2.2. Decisiones de diseño

La principal decisión de diseño fue utilizar tuplas como llaves en vez de strings concatenados (como "127.0.0.1:8885"). Esta decisión ofrece:

- Mejor eficiencia al evitar operaciones de concatenación de strings en cada consulta
- Tipo de dato inmutable apropiado para usar como llave de diccionario
- Mayor claridad en el código al mantener los componentes separados

2.3. Router Default

El router default debe colocarse al **final de la tabla de rutas** (última línea). Esto se debe a que las rutas se procesan en orden secuencial de arriba hacia abajo, y el router default debe actuar como una ruta "catch-all" que capture todos los paquetes que no coincidan con ninguna ruta más específica.

Si se colocara al inicio, todos los paquetes serían enviados al router default, haciendo inútiles las demás rutas específicas de la tabla.

2.3.1. Tabla de rutas con router default

Para el router R1 en la configuración de 5 routers, la tabla queda:

```
127.0.0.1/24 8882 8885 127.0.0.1 8882  
127.0.0.1/24 7000 7000 127.0.0.1 7000
```

La primera línea maneja el tráfico hacia la red conocida (puertos 8882-8885), mientras que la segunda línea captura cualquier otro tráfico y lo dirige al router default en el puerto 7000.

2.4. Pruebas: Mini-Internet sin TTL

2.4.1. Prueba 1: Tabla de rutas mal configurada

Al modificar la tabla de rutas del router R2 como se indica:

```
127.0.0.1/24 8881 8881 127.0.0.1 8881  
127.0.0.1/24 8883 8883 127.0.0.1 8881
```

Observaciones: Se produce un ciclo infinito. Cuando se envía un paquete desde R1 hacia R3:

1. R1 recibe el paquete y lo reenvía a R2 (según su tabla).

2. R2 recibe el paquete destinado a R3 (puerto 8883) y según su tabla mal configurada, lo reenvía de vuelta a R1.
3. R1 lo vuelve a enviar a R2.
4. El ciclo continúa indefinidamente hasta que el usuario interrumpe la ejecución.

Este comportamiento ilustra la importancia de configurar correctamente las tablas de rutas y anticipa la necesidad del campo TTL que se implementa posteriormente.

2.4.2. Prueba 2: Round-robin en red de 5 routers

Al enviar múltiples paquetes desde R1 hacia R5, se observó:

Cantidad de saltos: Los paquetes dan entre 3 y 4 saltos dependiendo de la ruta tomada. La ruta más corta es $R1 \rightarrow R2 \rightarrow R4 \rightarrow R5$ (3 saltos), mientras que la ruta alternativa es $R1 \rightarrow R2 \rightarrow R3 \rightarrow R5$ (4 saltos).

Variabilidad: No siempre dan la misma cantidad de saltos. Debido al algoritmo de round-robin implementado en R2, los paquetes alternan entre ser enviados a R3 (puerto 8883) y R4 (puerto 8884), resultando en caminos de diferente longitud.

Comparación con ruta óptima: La cantidad mínima de saltos posible es 3 (usando la ruta por R4). Sin embargo, debido al round-robin, aproximadamente la mitad de los paquetes toman la ruta por R3, resultando en 4 saltos. Esto representa un 33% más de saltos que el óptimo, lo que ilustra el trade-off entre balanceo de carga y eficiencia de ruta.

2.4.3. Prueba 3: Round-robin en red extendida (R0 y R6)

En la configuración extendida con routers R0 y R6, donde:

- R0 se conecta con R1 y R2
- R6 se conecta con R2 y R3

Al intercalar el envío de paquetes desde R1 a R5 y desde R5 a R1:

Observaciones: El router R2, que es el punto central de la red, mantiene correctamente estados de round-robin independientes para cada área de destino. Cuando se envían paquetes hacia R5, R2 alterna entre R3 y R4. Cuando se envían paquetes hacia R1, R2 alterna entre sus rutas hacia esa zona. Los contadores no se mezclan, confirmando que la implementación basada en tuplas (`dest_ip`, `dest_port`) funciona correctamente para múltiples áreas.

2.5. Pruebas: Mini-Internet con TTL

2.5.1. Prueba 1: Tabla mal configurada con TTL=10

Al repetir la prueba con la tabla mal configurada de R2, pero ahora con TTL=10:

Observaciones: El paquete ya no queda atrapado en un ciclo infinito. En su lugar:

1. El paquete viaja entre R1 y R2, decrementando su TTL en cada salto.
2. Después de 10 saltos, el TTL llega a 0.
3. El router que recibe el paquete con TTL=0 lo descarta e imprime: "Se recibió paquete con TTL 0".
4. El ciclo se rompe automáticamente.

Diferencias con la Prueba 1 de Parte I: La principal diferencia es que el sistema ahora tiene un mecanismo de protección contra ciclos infinitos. Mientras que en la Parte I el ciclo continuaba hasta intervención manual, con TTL el paquete es automáticamente descartado después de un número máximo de saltos. Esto previene la congestión de la red causada por paquetes perdidos que dan vueltas indefinidamente.

Este comportamiento demuestra la importancia del campo TTL en el protocolo IP real, donde valores típicos son 64 (Linux/Unix), 128 (Windows) o 254 (Solaris/AIX).

2.5.2. Prueba 2: Orden de llegada de paquetes

Al enviar un archivo de múltiples líneas desde R1 a R5 usando:

```
cat test_file.txt | python3 prueba_router.py 127.0.0.1;8885;10 127.0.0.1 8881
```

Observaciones sobre el orden:

Los paquetes **no necesariamente llegan en orden**. Se observaron los siguientes fenómenos:

1. **Reordenamiento por rutas diferentes:** Debido al round-robin en R2, algunos paquetes toman la ruta por R3 (4 saltos) y otros por R4 (3 saltos). Los paquetes que van por la ruta más corta pueden llegar antes que paquetes enviados anteriormente pero que tomaron la ruta más larga.
2. **No hay garantía de orden en UDP:** Los sockets UDP no orientados a conexión no garantizan orden de entrega. Incluso paquetes que toman la misma ruta pueden llegar en orden diferente debido a variaciones en el procesamiento.
3. **Impacto de round-robin:** El algoritmo de round-robin contribuye directamente al reordenamiento, ya que envía paquetes consecutivos por caminos de longitud diferente.

Relación con round-robin y múltiples rutas: El reordenamiento es una consecuencia directa del balanceo de carga. En redes reales, este comportamiento es común cuando se utiliza ECMP (Equal-Cost Multi-Path) o técnicas similares. Las aplicaciones que requieren entrega ordenada deben implementar sus propios mecanismos de reordenamiento en capas superiores (como TCP) o incluir números de secuencia en sus protocolos de aplicación.

3. Parte II: Fragmentación IP

3.1. Prueba 1: Validación de TTL y Round-Robin

Se envió un paquete pequeño desde R1 a R5 con los siguientes campos:

- IP: 127.0.0.1
- Puerto: 8885
- TTL: 10
- ID: 347
- Offset: 0
- Tamaño: 5
- FLAG: 0
- Mensaje: "hola!"

Comando ejecutado:

```
python3 send_message.py 127.0.0.1 8885 "hola!" 127.0.0.1 8881 10 347 0 5 0
```

Resultados: El código maneja correctamente el nuevo formato de headers. El paquete viaja desde R1 a R5, y se observa:

1. **TTL funciona correctamente:** El TTL se decrementa en cada salto como se esperaba.
2. **Round-robin se mantiene:** Al enviar múltiples paquetes, R2 continúa alternando entre R3 y R4.
3. **Sin fragmentación:** El tamaño total del paquete (23 bytes: 18 de header + 5 de mensaje) es menor que todos los MTUs de la red, por lo que no se fragmenta.

Estos resultados confirman que la funcionalidad de la actividad anterior se preservó correctamente al extender el sistema con los nuevos campos de header.

3.2. Prueba 2: Fragmentación y Re-ensamblaje

Se creó un paquete de tamaño total 150 bytes (18 bytes de header + 132 bytes de mensaje) y se envió desde R1 hacia R5.

Configuración de MTUs en la red:

- R1 → R2: MTU = 100 bytes
- R2 → R3: MTU = 50 bytes
- R2 → R4: MTU = 80 bytes
- R3 → R5: MTU = 55 bytes
- R4 → R5: MTU = 140 bytes

3.2.1. Comportamiento observado desde R1 a R5

Primera fragmentación en R1: El paquete original de 150 bytes es fragmentado en R1 antes de enviarlo a R2:

- Fragmento 0: 100 bytes (18 header + 82 datos), offset=0, flag=1
- Fragmento 1: 68 bytes (18 header + 50 datos), offset=82, flag=0

Re-fragmentación en R2: Cuando los fragmentos llegan a R2, este decide la ruta según round-robin:

Si va por R3 (MTU=50):

- Fragmento 0 (100 bytes) se re-fragmenta en:
 - Sub-fragmento 0a: 50 bytes (32 datos), offset=0, flag=1
 - Sub-fragmento 0b: 50 bytes (32 datos), offset=32, flag=1
 - Sub-fragmento 0c: 36 bytes (18 datos), offset=64, flag=1
- Fragmento 1 (68 bytes) se re-fragmenta en:
 - Sub-fragmento 1a: 50 bytes (32 datos), offset=82, flag=1
 - Sub-fragmento 1b: 36 bytes (18 datos), offset=114, flag=0

Si va por R4 (MTU=80):

- Fragmento 0 (100 bytes) se re-fragmenta en:
 - Sub-fragmento 0a: 80 bytes (62 datos), offset=0, flag=1
 - Sub-fragmento 0b: 38 bytes (20 datos), offset=62, flag=1
- Fragmento 1 (68 bytes) pasa sin fragmentar, offset=82, flag=0

Re-ensamblaje en R5: R5 recibe todos los fragmentos (posiblemente en desorden) y los almacena en su buffer, indexados por el ID del paquete. Cuando detecta que tiene todos los fragmentos (verificando que hay un fragmento con flag=0 y que los offsets son contiguos), procede a reensamblarlos y muestra el mensaje completo.

3.2.2. Cambio de comportamiento según router origen

Al enviar desde otros routers, el comportamiento cambia debido a las diferentes topologías de ruta:

Desde R4 a R5: La ruta es directa con MTU=140, por lo que un paquete de 150 bytes solo se fragmenta una vez en 2 fragmentos que llegan directamente a R5.

Desde R3 a R5: Con MTU=55, un paquete de 150 bytes se fragmenta en múltiples fragmentos pequeños que llegan directamente a R5.

3.2.3. Capacidad de re-fragmentación

Confirmación: Sí, el código logra re-fragmentar un fragmento en fragmentos más pequeños cuando es necesario. Esto se observó claramente cuando los fragmentos creados en R1 (con MTU=100) fueron re-fragmentados en R2 al tomar la ruta por R3 (MTU=50).

La función `fragment_IP_packet()` opera sobre cualquier paquete IP válido, sin distinguir si es un paquete original o un fragmento. Cuando recibe un fragmento, simplemente:

1. Verifica si su tamaño excede el MTU.
2. Si lo excede, divide el contenido del mensaje en trozos más pequeños.
3. Ajusta los campos offset y flag apropiadamente para cada sub-fragmento.
4. Mantiene el mismo ID para que puedan ser reensamblados correctamente en destino.

Esta capacidad es fundamental en Internet real, donde un paquete puede pasar por enlaces con MTUs cada vez más restrictivos, requiriendo fragmentación sucesiva.

4. Conclusiones

4.1. Forwarding Básico

La implementación de forwarding básico demostró exitosamente los conceptos fundamentales de enrutamiento en redes:

1. Las tablas de rutas son el mecanismo fundamental para tomar decisiones de forwarding en routers.
2. El algoritmo round-robin proporciona un balanceo de carga simple pero efectivo, aunque a costa de potencialmente no utilizar siempre la ruta más corta.
3. El campo TTL es esencial para prevenir ciclos infinitos causados por configuraciones erróneas o transitorias de las tablas de rutas.
4. Los protocolos no orientados a conexión como UDP no garantizan orden de entrega, especialmente cuando se combinan con técnicas de balanceo de carga que utilizan múltiples rutas.

4.2. Fragmentación IP

La extensión con fragmentación IP añadió complejidad significativa pero necesaria:

1. El concepto de MTU es fundamental en redes heterogéneas donde diferentes enlaces tienen diferentes capacidades.
2. La fragmentación y re-ensamblaje permiten que datagramas grandes atraviesen enlaces con MTUs restrictivos sin necesidad de que la fuente conozca todos los MTUs del camino.
3. La capacidad de re-fragmentación es crucial, ya que un paquete puede necesitar ser fragmentado múltiples veces a medida que atraviesa enlaces con MTUs progresivamente más pequeños.
4. El re-ensamblaje solo ocurre en el destino final, no en routers intermedios, lo que simplifica el procesamiento en los routers pero requiere que el destino tenga suficiente memoria para almacenar fragmentos.

4.3. Reflexión Final

Este proyecto proporcionó una comprensión práctica profunda de los mecanismos de la capa de red. La implementación desde cero de conceptos como forwarding, TTL, round-robin y fragmentación permitió apreciar:

- La complejidad inherente en el enrutamiento de paquetes
- Los trade-offs entre eficiencia y balanceo de carga
- La importancia de mecanismos de protección como TTL
- Los desafíos de manejar redes heterogéneas con diferentes MTUs

La experiencia de depurar problemas como ciclos de enrutamiento y reordenamiento de paquetes reforzó la comprensión de por qué los protocolos de red están diseñados de la forma en que están, y por qué capas superiores como TCP son necesarias para proporcionar garantías adicionales como entrega ordenada y confiable.